

[Paper] LoRA: Low-Rank Adaptation of Large Language Models

Posted Sep 10, 2026 • Updated Sep 10, 2026

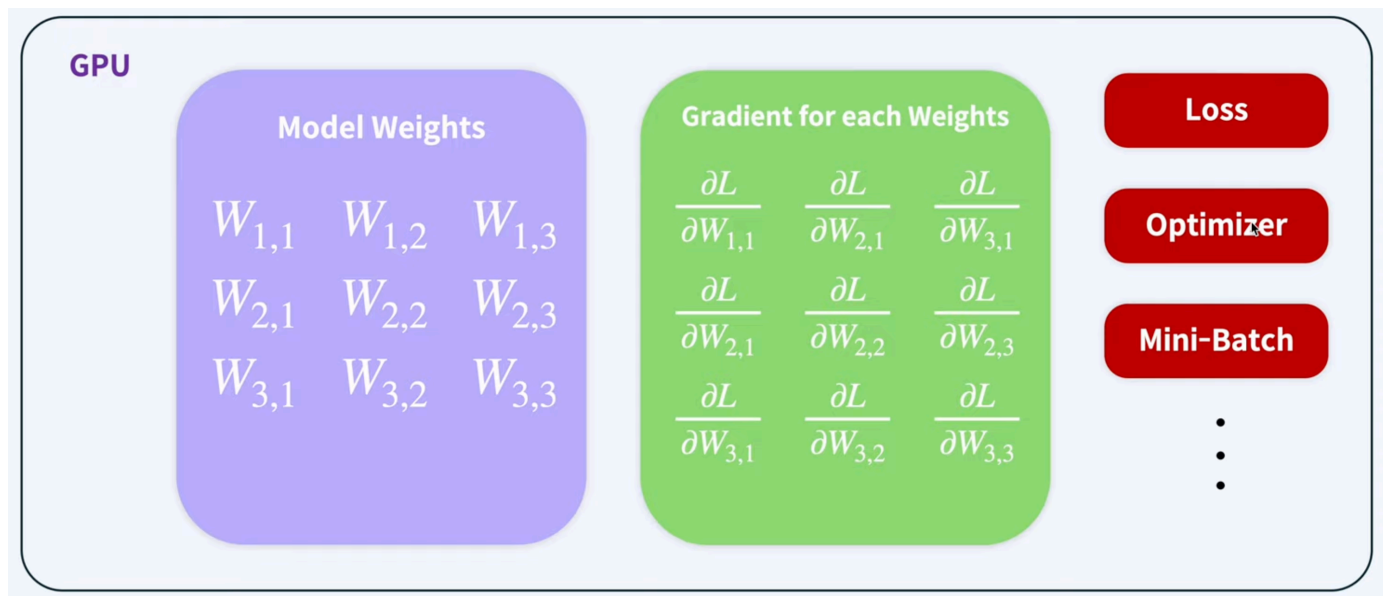
By 김민준 (Min-jun, Kim)

Contents >

Introduction

LoRA는 거대 인공지능 모델의 전체 가중치를 수정하는 대신, 적은 수의 추가 파라미터만 학습시켜 비용과 시간을 크게 줄이는 고효율 파인튜닝 기법이다.

Model Size가 커졌고, 대형언어모델은 GPU에 올리는 것조차 쉽지 않음.



Model Weights 뿐만 아니라 Model Weights를 훈련하는 모든 파라미터에 대해서 미분값도 함께 저장하게 됨. Loss Optimizer Mini-Batch 등 추가적으로 저장해야하는 것들도 많음.

즉, Fine-tuning 할 때 GPU에 많은 자원을 올려야함.

Main Goal

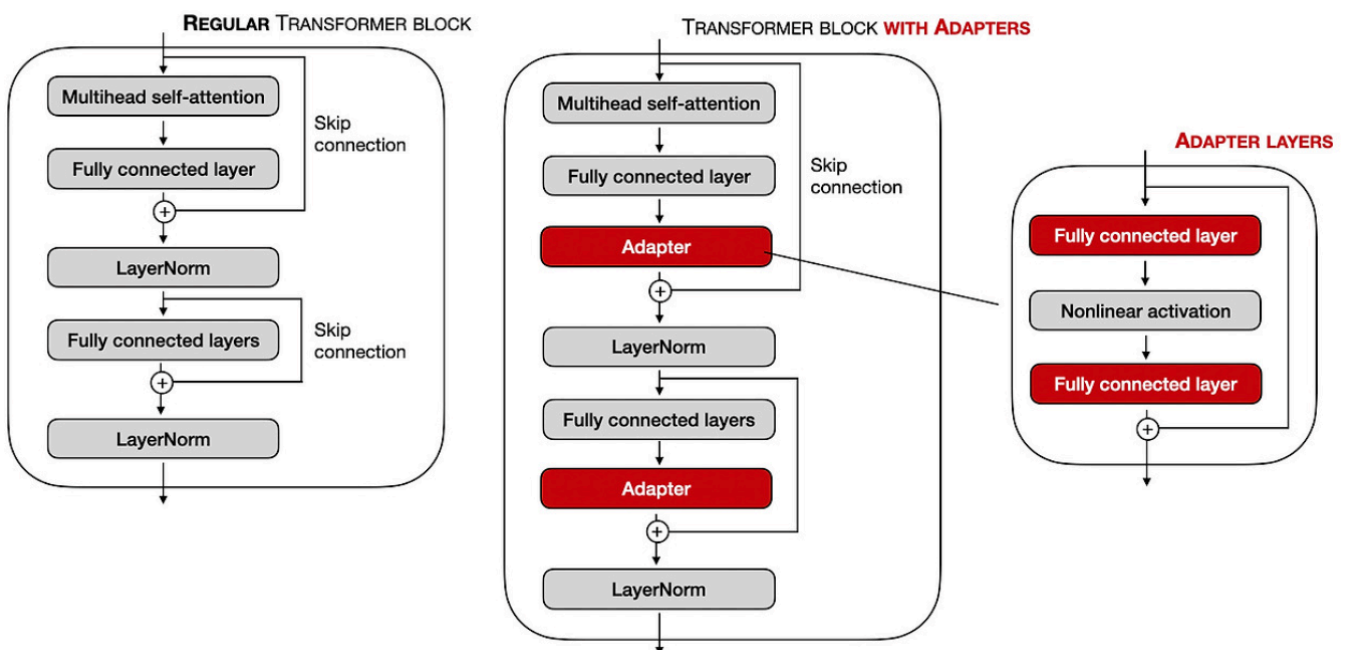
대규모 사전학습 모델의 weight는 고정하고, task adaptation에 필요한 weight update만 low-rank 형태로 학습하여 Fine-tuning 비용을 줄이는 것.

Motivation: Limitations of Full Fine-Tuning

- 사전학습된 모델의 (일반화)성능을 떨어뜨릴 수 있음.
- large scale의 labeled data가 필요함.
- High optimization complexity and time-consuming

Existing Parameter-Efficient Fine-Tuning Methods

Adapter Layers



큰 모델의 기존 파라미터는 거의 그대로 두고, 중간중간에 작은 학습용 모듈(Adapter)을 추가해서 그것만 학습하는 방식

Adapter 자체는 보통 이런 작은 bottleneck 구조임. $h \rightarrow W_{\text{down}} \rightarrow \text{활성화함수} \rightarrow W_{\text{up}} \rightarrow h$
(Bottleneck: 인공신경망에서 데이터의 차원(채널)을 강제로 줄였다가 다시 늘리는 계층(Layer)을 의미함.)

학습할 때는 원래 Transformer의 수억~수십억 개 파라미터를 freeze하고, 이 작은 Adapter 파라미터만 업데이트함.

Modern GPU에서는 효율(속도)가 좋지 않음. 학습하는 파라미터는 적어지지만, inference 때도 이 Adapter를 계속 통과해야 함. 즉 연산 단계가 추가됨. (모델의 깊이를 확장하여 inference latency를 도입함)

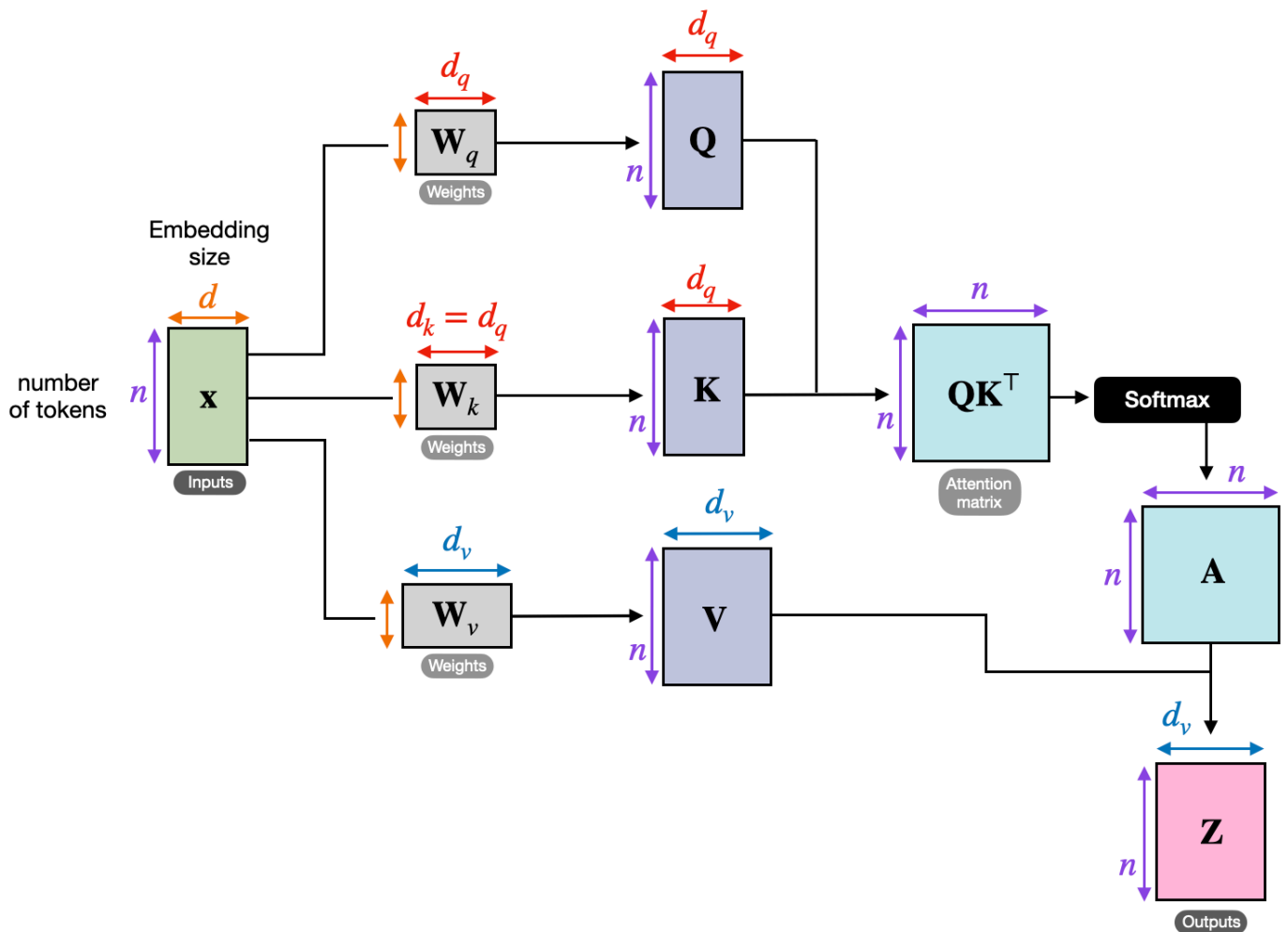
Prefix Tuning

혹은 이와 다르게 모델의 사용 가능한 시퀀스 길이를 줄이는 방법도 존재한다. (prefix tuning)

위와 같은 기존 PEFT는 저장공간은 아꼈지만 Adapter는 느려지고, Prefix는 context를 잡아먹고, Full Fine-Tuning보다 성능도 낮았다. 파라미터는 훨씬 적게 학습하지만, 성능이 떨어진다는 trade-off가 존재.

Method

LoRA가 어떻게 Adapter처럼 시간복잡도(연산량)을 늘리지 않으면서도 효과적으로 fine-tuning 할 수 있게끔 만들었는지 알아보자.

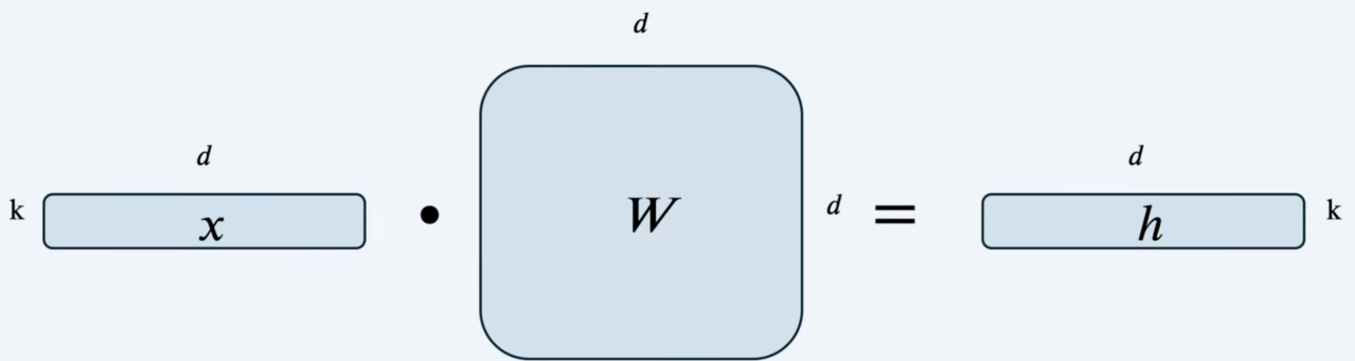


Problem Formulation

Φ 로 parameterize된 사전 학습된 autoregressive model $P_{\Phi}(y|x)$ 가 주어졌다고 가정

LoRA는 W_q, W_k, W_v, W_o 와 같은 기존 **weight**를 **freeze**하고, 해당 **weight**에 더해질 **low-rank update** $\Delta W = BA$ 만 최적화한다.

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log (P_{\Phi}(y_t|x, y_{<t}))$$

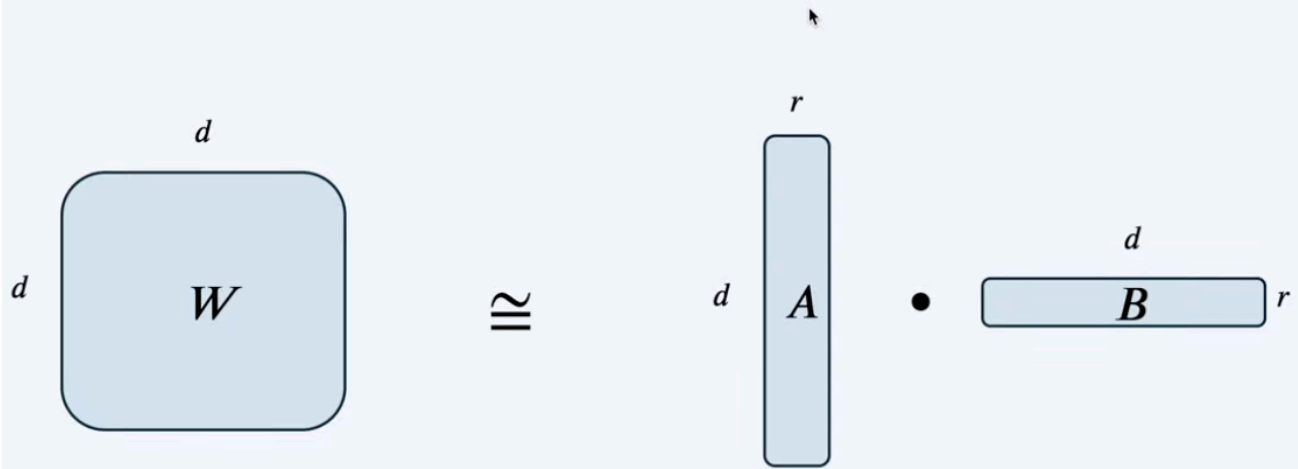


언어 모델의 학습 목적함수, 정확히는 Maximum Likelihood Objective 임.

Φ 는 모델 전체 파라미터이고, W 는 그 안에 들어있는 여러 weight matrix 중 하나임.

입력 x 와 이전 토큰들을 보고 다음 정답 토큰을 높은 확률로 맞히도록 모델을 학습하는 식이다.

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y_{<t})) \quad r < \min(k, d)$$



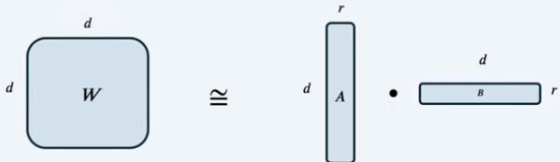
Low-Rank-Parameterized Update Matrices

Full Fine-tuning에서는 기존 가중치 W_0 전체를 업데이트한다.

하지만 LoRA는 downstream task에 적응하기 위해 필요한 변화량 ΔW 로는 낮은 rank를 가질 수 있다고 가정한다.

Full Fine-tuning에서 필요할 weight update ΔW 를 low-rank 형태 $B \cdot A$ 로 근사한다.

선형종속적인 관계는 제외시키고, 적은 차원의 weight matrix만 쓰더라도 충분히 W 를 근사하게 표현할 수 있다라는 것을 깔고 감.



where $d = 4096$ and $r = 16$

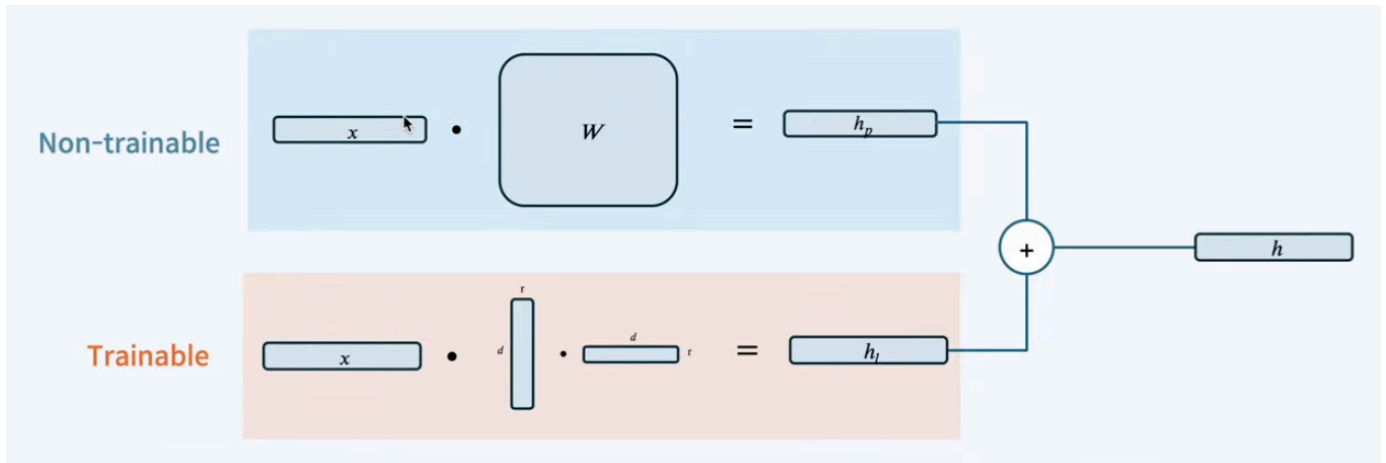
Parameter of W : **$4096 \times 4096 = 16,777,216$**

A : **$4096 \times 16 = 65,536$**

B : **$16 \times 4096 = 65,536$**

Total: **$131,072$ (0.78%)**

Forward Computation



frozen pretrained path와 trainable low-rank path의 출력을 더해 최종 hidden representation h 를 계산한다.

Training and Inference

Training



LoRA Weights 가 추가되지만, 학습 시 Gradient of LoRA weight만 Gradient로 넣어주면 되기 때문에 더 작은 weight만을 효과적으로 학습함.

Inference and Weight Merging

Inference 시, LoRA Weights 까지 같이 GPU에 올려야 해서 추가메모리와 추가연산이 존재함. (순수 inference와 비교 시)

하지만 LoRA는 Adapter와 달리 기존 weight와 병렬적인 low-rank branch로 학습되기 때문에 모델 깊이를 추가하지 않는다.

그리고 inference 시에는 이 low-rank update를 기존 weight에 merge할 수 있으므로, merge 후에는 별도의 추가 연산 없이 inference할 수 있다.

$$W_0 \in \mathbb{R}^{d \times k} \quad \gamma = \begin{bmatrix} \gamma_1 \\ \gamma_2 \\ \vdots \\ \gamma_k \end{bmatrix} \in \mathbb{R}^{k \times 1}$$

$$W = W_0 + \Delta W$$

$$\Delta W = B \cdot A$$

$B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}$
 $(r \ll \min(d, k))$

$r(k+d)$ 개만 학습.

Full Fine-tuning

non-trainable
trainable

LoRA: $h = W_0 \gamma + BA\gamma$

(실제) $h = W_0 \gamma + \left(\frac{\alpha}{r}\right) BA\gamma$

Scaling

r : low-rank 중간 차원
 α : LoRA 보정값의 세기를 조절하는 scaling hyperparameter

Inference

① LoRA 분

$h = W_0 x + \frac{\alpha}{r} B A x$

대조 계산.

GPU: $W_0 + A + B$ 가 필요. 추가 연산 0.

② Merged

$W_{\text{merged}} = W_0 + \frac{\alpha}{r} B A$

$h = W_{\text{merged}} \cdot x$ 한 계산

Experiments

Comparison with Fine-Tuning Baselines

근사를 하게 되면 성능이 떨어지는 것이 보편적이지만, 일반적인 Full Fine-tuning 보다도 LoRA가 성능도 일관적이고 성능이 좋은 경우가 다수.

Fine-tuning 을 하게 되면 기존의 weights가 일부 사라지기 때문에, 일반화 성능이 떨어질 수 있음.

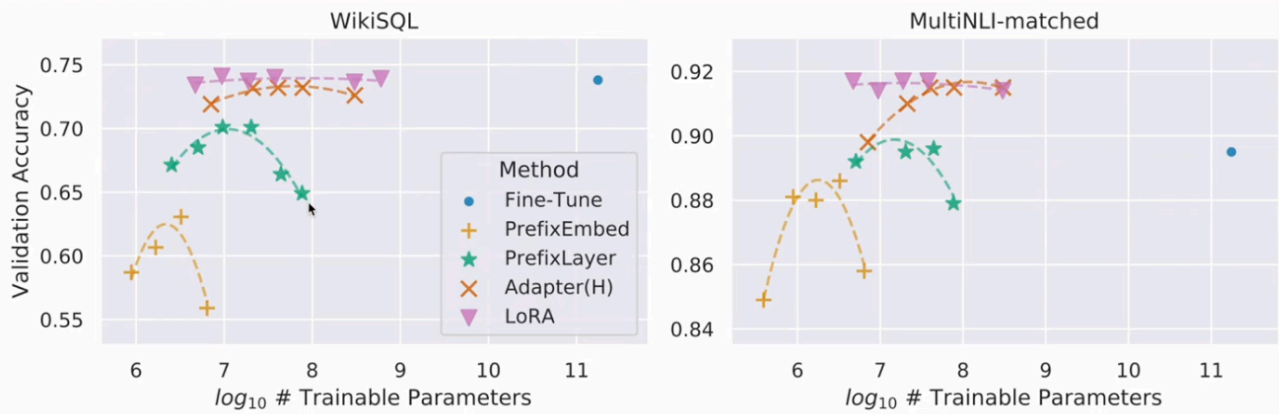


Figure 2: GPT-3 175B validation accuracy vs. number of trainable parameters of several adaptation methods on WikiSQL and MNLI-matched. LoRA exhibits better scalability and task performance. See Section F.2 for more details on the plotted data points.

LoRA Fine-tuning은 기존 weight는 변화시키지 않고 LoRA Weight만 변화시키기 때문에 성능이 더 좋아질 수 있음을 알 수 있음.

Which Weight Matrices Should LoRA Be Applied To?

	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

Table 5: Validation accuracy on WikiSQL and MultiNLI after applying LoRA to different types of attention weights in GPT-3, given the same number of trainable parameters. Adapting both W_q and W_v gives the best performance overall. We find the standard deviation across random seeds to be consistent for a given dataset, which we report in the first column.

What Is the Optimal Rank r for LoRA?

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

Table 6: Validation accuracy on WikiSQL and MultiNLI with different rank r . To our surprise, a rank as small as one suffices for adapting both W_q and W_v on these datasets while training W_q alone needs a larger r . We conduct a similar experiment on GPT-2 in Section H.2.

실험 결과, LoRA는 높은 rank가 반드시 필요하지 않았으며, 특히 q W_v 에 함께 적용할 경우 매우 작은 rank만으로도 충분한 downstream adaptation 성능을 얻을 수 있었다. 이는 fine-tuning에서 필요한 weight update가 실제로 low-rank 구조를 가진다는 LoRA의 가정을 뒷받침한다.

 [Paper](#)

This post is licensed under CC BY 4.0 by the author.

Share:    

OLDER

[\[대외활동\] 멋쟁이사자처럼·백엔드 크로스오버
과제 API 설계 고민기록 \(Spring Boot\)](#)

NEWER

-